

Algorithm for file updates in Python

Project description

In this project I created a python algorithm for my organization that uses Python code to check whether the allow list contains any IP addresses identified on the remove list. If so, I should remove those IP addresses from the file containing the allow list.

Open the file that contains the allow list

The file that I want to open is called "allow_list.txt". I Assigned a string containing this file name to the import_file variable. Then, use a with statement to open it.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a List of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First Line of `with` statement
with open(import_file, "r") as file:
```

Read the file contents

Next, I used the .read() method to convert the contents of the allow list file into a string so that I can read them. Then I stored this string in a variable called ip_addresses.

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a List of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Display `ip_addresses`
print(ip_addresses)
```

Convert the string into a list

In order to remove individual IP addresses from the allow list, the IP addresses need to be in a list format. Therefore, I used the .split() method to convert the ip_addresses string into a list.

```

# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Display `ip_addresses`
print(ip_addresses)

```

Iterate through the remove list

A second list called `remove_list` contains all of the IP addresses that should be removed from the `ip_addresses` list. I set up the header of a for loop that will iterate through the `remove_list`. Use `element` as the loop variable.

```

# Build iterative statement
# Name Loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Display `element` in every iteration
    print(element)

```

Remove IP addresses that are on the remove list

I added code that will remove all the IP addresses from the allow list that are also on the remove list. First, there is a conditional that evaluates if the loop variable `element` is part of the `ip_addresses` list. Then, within that conditional, the `.remove()` method is applied to the `ip_addresses` list and then the IP addresses identified in the loop variable `element` will be removed.

```

for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Display `ip_addresses`
print(ip_addresses)

```

Update the file with the revised list of IP addresses

Now that these IP addresses from the `ip_address` variable are removed, I completed the algorithm by updating the file with this revised list. To do this, I converted the `ip_addresses` list back into a string using the `.join()` method. Then applied `.join()` to the string `"\n"` in order to separate the elements in the file by placing them on a new line.

Then, I used another `with` statement and the `.write()` method to write over the file assigned to the `import_file` variable.

```
# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`
    file.write(ip_addresses)
```

Summary

I developed an algorithm to remove specific IP addresses listed in the `remove_list` variable from the `allow_list.txt` file, which contains approved IP addresses. The process involved opening the file, reading its contents as a string, and then converting this string into a list stored in the `ip_addresses` variable. I then iterated through each IP address in `remove_list`, checking whether it existed in the `ip_addresses` list. If a match was found, I used the `.remove()` method to eliminate it from `ip_addresses`. Afterward, I utilized the `.join()` method to convert the updated `ip_addresses` list back into a string and overwrite the `allow_list.txt` file with the revised list of IP addresses.